

00-实验0：开发环境搭建

实验0：开发环境搭建与工具链初探

实验目标

本实验旨在帮助大家搭建基于 RISC-V 架构的操作系统开发环境，并理解各个基础工具在后续操作系统开发中的作用。在真正的硬件开发中，将代码烧录到物理主板上调试非常耗时且繁琐，因此我们将使用 QEMU 模拟器来充当我们的“主板和 CPU”。

核心概念：为什么需要这些工具？

在平时的 C 语言编程课中，我们编写 C 代码直接用 `gcc` 编译就能在自己的电脑上跑。但在开发操作系统时，环境完全不同，为了跨越这第一道鸿沟，我们需要认识以下三件法宝：

- 交叉编译工具链 (Cross-Compilation)**：我们是在一台 x86 架构（或 ARM 架构）的个人电脑上写代码，但编译出来的机器指令必须能在 RISC-V 架构的 CPU 上运行。因此我们需要特殊的编译器（如 `riscv64-unknown-elf-gcc`）。
- 硬件模拟器 (Emulator)**：我们没有真实的 RISC-V 物理主板，因此使用 QEMU 软件来虚拟出一个包含 CPU、内存、串口等硬件设备的运行环境。
- 多架构调试器**：操作系统的 bug 往往涉及最底层的硬件寄存器、内存绝对地址，普通的调试方法不再奏效。我们需要 `gdb-multiarch` 结合 QEMU 进行汇编指令级的查错。

环境搭建步骤

1. 安装基础依赖与模拟器

我们首先需要安装构建工具（如 `make`）和 QEMU 模拟器等基础软件。

```
1 # Ubuntu/Debian系统（如果你使用 Windows，请先安装 WSL2 Ubuntu 22.04）
2 sudo apt-get update
3 sudo apt-get install -y build-essential git python3 qemu-system-misc expect gdb-multiarch
```

✅ 验证 QEMU：

```
1 qemu-system-riscv64 --version
```

预期效果：终端应输出 QEMU emulator version 5.0.0 或更高版本号。如果提示 `command not found`，请检查安装步骤。

2. 安装 RISC-V 交叉编译工具链

这个工具链的全名很长：`riscv64` 代表 64 位 RISC-V 架构，`unknown` 表示不需要依赖特定的底层操作系统（我们正在写操作系统本身！），`elf` 则是目标文件的标准格式。

方案 A：使用预编译包（🏆 推荐，速度快且稳定）

```
1 wget https://github.com/riscv-collab/riscv-gnu-toolchain/releases/download/2023.07.07/riscv64-elf-ubuntu-20.04-gcc-nightly-2023.07.07-nightly.tar.gz
2 sudo tar -xzf riscv64-elf-ubuntu-20.04-gcc-nightly-2023.07.07-nightly.tar.gz -C /opt/
3 # 将工具链路径加入当前用户的环境变量
4 echo 'export PATH="/opt/riscv/bin:$PATH"' >> ~/.bashrc
5 source ~/.bashrc
```

方案 B：包管理器安装（如果你的包管理器自带的版本可用）

```
1 sudo apt-get install gcc-riscv64-unknown-elf
```

✅ 验证工具链：

```
1 riscv64-unknown-elf-gcc --version
```

预期效果：终端应输出 `gcc` 的版本信息（如 `10.x` 或以上）。如果找不到命令，请检查上一条方案 A 中环境变量配置是否生效。

3. 创建极简测试验证工具链

光能打出版本号还不够，我们来真实地编译一个最简单的 RISC-V 程序，测试交叉编译是否畅通。

```
1 # 1. 写一段最简单的 C 代码
2 echo 'int main(){ return 0; }' > test.c
3
4 # 2. 使用交叉编译器将它编译为 RISC-V 架构的对象文件
5 riscv64-unknown-elf-gcc -c test.c -o test.o
6
7 # 3. 查看生成的文件属性
8 file test.o
```

预期效果：终端应输出类似 `test.o: ELF 64-bit LSB relocatable, UCB RISC-V...` 的信息。只要在这里看到了 `RISC-V 64-bit` 字眼，就说明你的交叉编译工具链已经大功告成，顺利产出了可以在目标架构上跑的代码！

4. 获取参考资料与创建项目结构

在后续的实验中，我们将以著名的 MIT xv6 操作系统为重要参考蓝本。现在，建立我们自己的实验领地。

```
1 # 克隆 xv6 源码作为后续实验的参考字典（不直接在其上修改）
2 git clone https://github.com/mit-pdos/xv6-riscv.git
3 cd xv6-riscv && make qemu # (可选) 验证 xv6 源码能否在你的环正常运行, Ctrl-
  A X 退出 QEMU
4
5 # 回到上级目录, 创建你自己的操作系统项目文件夹
```

```
6 cd ..
7 mkdir my-riscv-os && cd my-riscv-os
8 git init
9 mkdir -p kernel/{boot,mm,trap,proc,fs,net} include scripts
```

实验总结与自查指引

搭建环境往往是劝退的最主要的一步。遇到问题，请首先自查：

1. 运行工具链命令提示找不到时，`PATH` 环境变量中是否包含了 `/opt/riscv/bin`？可以通过 `echo $PATH` 检查。
2. Windows 用户强烈建议开启 WSL2 (推荐 Ubuntu 22.04) 来执行上述指令操作，切勿在传统的 Windows CMD 下尝试。
3. Apple Silicon (M1/M2/M3) 的 Mac 用户可以通过 Homebrew 安装对应的依赖：`brew install qemu riscv-gnu-toolchain`。